

The Trino Optimization Handbook



```
SELECT id, name AS product, quality,  
       technique  
FROM market  
WHERE use_case = "Trino Optimization"  
ORDER BY quality DESC  
LIMIT 1
```

id	product	quality	technique
1	Alluxio	100%	caching

Best Practices & Tuning Tips

INTRODUCTION

In today's data-driven world, having a high-performance data querying platform is crucial for efficient data analysis, rapid insights, and informed decision-making. Trino (formerly known as PrestoSQL), a popular open-source distributed SQL query engine, has become an indispensable tool for organizations seeking to run analytics across various data sources.

However, many Trino users and developers still face challenges with inconsistent query performance. Understanding Trino's intricacies and performance optimization techniques is essential for organizations aiming to derive maximum value from their data. Fast queries not only enhance user experience but also lead to quicker insights, improved resource efficiency, and reduced costs.

This guide aims to equip data platform engineers, data engineers, Trino administrators, developers, and advanced users with the knowledge and best practices necessary to maximize Trino query performance. You will learn a comprehensive approach for Trino optimization, covering data formats (Parquet, ORC), table formats (column layouts), data sources (S3 and more), and data platform architecture related to Trino performance.

By the end of this ebook, you will have a deeper understanding of the following:

- How Trino runs queries under the hood
- What happens during query execution and the bottlenecks that can impact query performance
- How to refine Trino for optimal query performance
- The tuning steps and seven best practices for maximizing Trino query efficiency, including configuration settings, session properties, SQL statement and practical advice
- Real-world examples of Trino optimizations using caching

So, let's get started and unlock the full potential of Trino for your data analytics needs!

Table of Contents

Chapter 1: Trino Architecture & Key Concepts for Optimization **4**

<u>1.1 A Recap of Trino Architecture and Components</u>	5
<u>1.2 Trino Query Execution Model and How It Relates to Optimization</u>	6
<u>1.2.1 Trino Coordinator</u>	7
<u>1.2.2 Trino Worker</u>	8
<u>1.3 The Importance of Optimization in Trino</u>	9

Chapter 2: Best Practices for Trino Optimization11

<u>2.1 Identify Bottlenecks Using EXPLAIN and EXPLAIN ANALYZE</u>	12
<u>2.2 Determine the Max Concurrency and Max Memory Per Query</u>	14
<u>2.3 Optimizing File Formats and Table Layouts</u>	16
<u>2.3.1 Columnar Data File Formats and Compression</u>	17
<u>2.3.2 Partitioning and Bucketing Strategies</u>	18
<u>2.3.3 Utilizing Materialized Views</u>	19
<u>2.4 Gathering and Using Hive Table Statistics</u>	20
<u>2.5 Join Optimization Strategies</u>	21
<u>2.5.1 Types of Join Distributions</u>	22
<u>2.5.2 Optimizing Join Orders</u>	23
<u>2.5.3 Implementing Dynamic Filtering</u>	24
<u>2.6 Data Caching for Trino</u>	25
<u>2.7 Recommended JVM Configuration Settings</u>	27

Chapter 3: Real-World Case: Optimize Trino with Caching 19

<u>3.1 Shopee: Enhancing Query Performance in Large-scale Deployments</u>	30
<u>3.2 Razorpay: Adapting Trino Clusters for Elasticity</u>	31

Chapter 4: Key Takeaways & Resources for Further Learning.... **33**

<u>4.1 Key Takeaways</u>	34
<u>4.2 Additional Resources</u>	35

CHAPTER 01 Trino Architecture and Key Concepts for Optimization

In this chapter, we will first provide an overview of Trino's high-level architectural components to help you understand how Trino works before diving into query optimization. Next, we will discuss the query lifecycle and execution model of Trino in detail, which is essential for diagnosing and tuning slow queries.



Cluster



Connector



Coordinator



Worker



Catalog

1.1

A Recap of Trino Architecture and Components

Trino is a distributed query engine that processes data in parallel across multiple servers, consisting of a single coordinator and multiple workers. Below is a brief recap of key components in Trino's architecture.

- **Cluster:** A Trino cluster comprises a coordinator and many workers. The coordinator manages query execution while the workers access data sources configured in catalogs and process each query in parallel.
- **Coordinator:** The coordinator is responsible for parsing SQL statements, planning queries, and managing Trino worker nodes. It creates a logical and physical plan of a query involving a series of stages and tasks running on a cluster of Trino workers.
- **Worker:** A Trino worker executes tasks and processes data, fetching data from data storage via connectors and exchanging intermediate data with other workers. The coordinator collects results from the workers and returns the final results to the client.
- **Connector:** A connector adapts Trino to a data source, like Hive data warehouse, data lake or even an relational database, allowing Trino to interact with a resource using a standard API. Trino contains dozens of built-in connectors, with many third-party developers contributing additional connectors for a variety of data sources.
- **Catalogs:** Trino accesses data through connectors, where each data source is configured as a catalog. You can query multiple catalogs within a single query using SQL joins. Catalogs are configured using properties files stored in the Trino configuration directory. For instance, it is a common use case to add a Hive catalog to connect a Hive data source, which can be done by Trino administrators.

1.2

Trino Query Lifecycle and How It Relates to Optimization

Understanding the query execution model is crucial for tuning Trino's performance. Let's take a look at the Trino query life cycle.

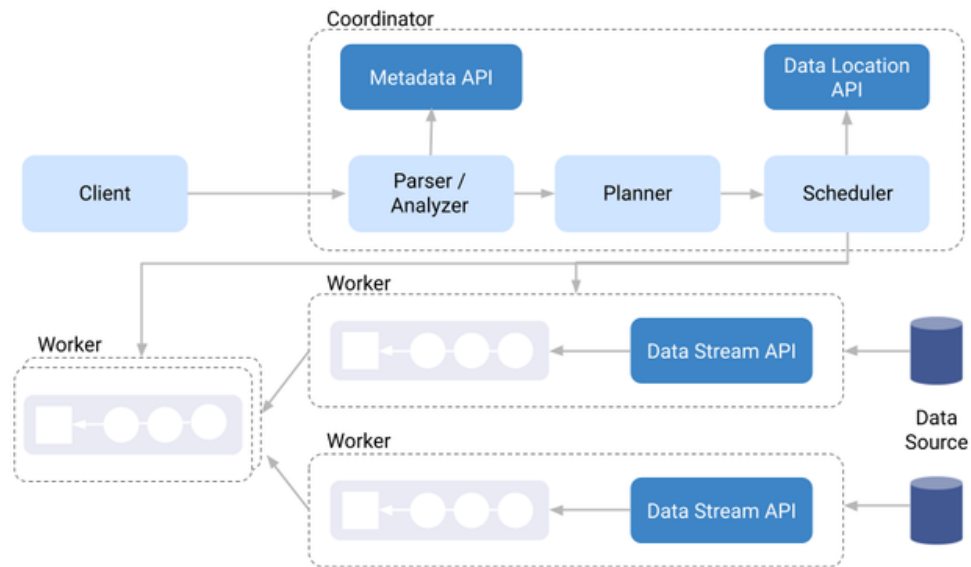


Figure 1. Trino Architecture. (R.Sethi, M.Traverso, et.al, 2019)

1.2.1

Trino Coordinator

The Trino coordinator is responsible for parsing SQL statements, planning queries, and returning the results to the client. You can also configure the coordinator to act as a Trino worker to participate in SQL execution, which might be helpful in small Trino clusters.

When a SQL statement is submitted to the coordinator, it parses, analyzes, and creates a query plan. For common table layouts in analytical workloads, such as Hive, Iceberg, and Hudi tables, Trino obtains metadata and data statistics from third-party services or external storage. The coordinator uses this metadata and data statistics to generate the query plan and create a distributed query plan, facilitating parallel processing across Trino workers.

Metadata reflects the table layout, while data statistics reflect data distribution. Both significantly impact query plan generation, which in turn affects query performance. In this book, we will discuss optimizing table layout and gathering statistics to improve query performance.

A distributed query plan consists of one or more stages, each containing one or more tasks. Tasks process data in the form of splits, with the coordinator assigning tasks from a stage to Trino workers in the cluster. Specific operations on the data depend on the underlying data source and connector. Trino workers execute these operations and pass the results to other workers for further processing or return the final results to the coordinator, which then sends them to the client.

1.2.2

Trino Worker

The Trino worker is responsible for reading data from external storage, processing data using SQL operations, exchanging data with other Trino workers, and aggregating data to produce the final query results.

Trino is a storage-independent compute engine, meaning it always loads data from external storage. The size of data files and the ability to skip unnecessary data reads are essential in Trino. Additionally, using faster storage, better network bandwidth, locating storage closer to the Trino cluster, or using a cache can also speed up data reading. Subsequent chapters will discuss best practices for improving data access efficiency, such as using compression, partitioning tables, using columnar data formats like ORC and Parquet, and applying proper predicate pushdown.

Trino workers provide computing power and are responsible for loading data for the Trino cluster. In most cases, equipping each worker with more and stronger CPUs will speed up queries. Furthermore, since Trino is storage-independent, faster access to external storage also speeds up query execution. For some operations, such as join, Trino needs to exchange data with other workers. In these cases, good network bandwidth is highly beneficial.

Memory is another crucial resource. By default, Trino uses memory instead of disks to store intermediate operation results. In daily operation and maintenance, you can consider the memory of all workers as a distributed memory resource pool. The pool's size determines the Trino cluster's processing and concurrency capabilities for queries. If a query's memory usage exceeds the total memory pool, the query will fail or be blocked indefinitely. When multiple queries run simultaneously, it's essential to balance the maximum memory usage per query and the maximum concurrency. We will discuss this in detail in later chapters.

1.3

The Importance of Optimization in Trino

Optimization plays a critical role in the performance and efficiency of any Trino deployment. Some key reasons why optimization is essential in Trino are:

- **Query performance:** Optimizing Trino allows for faster query execution, reducing the overall time taken to process queries. This can lead to significant productivity improvements, particularly when dealing with large-scale data processing tasks.
- **End-user experience:** Enhanced optimization results in a smoother end-user experience by ensuring faster response times and reducing query latency. This contributes to user satisfaction and enables more effective data analysis.
- **Resource utilization:** Efficient optimization enables Trino to use resources better, including CPU, memory, network and storage. This helps minimize the hardware footprint and associated costs while ensuring queries are processed as quickly as possible.
- **Scalability:** An optimized Trino deployment can handle increasing data volumes and query complexity better, ensuring that performance remains consistent as the system grows. This is especially important in today's data-driven world, where organizations continually process larger amounts of data and demand faster insights.
- **Concurrency:** Proper optimization can improve Trino's ability to handle concurrent queries, enabling multiple users to access the system simultaneously without experiencing performance degradation. This is crucial for maintaining a responsive and efficient environment, particularly in multi-user or multi-tenant deployments.
- **Cost savings:** By optimizing Trino, organizations can reduce infrastructure costs and save operation costs because of higher data engineer productivity. Efficient resource utilization and improved performance also contribute to these cost savings, helping organizations achieve a better return on their investment.



Chapter 1 Summary

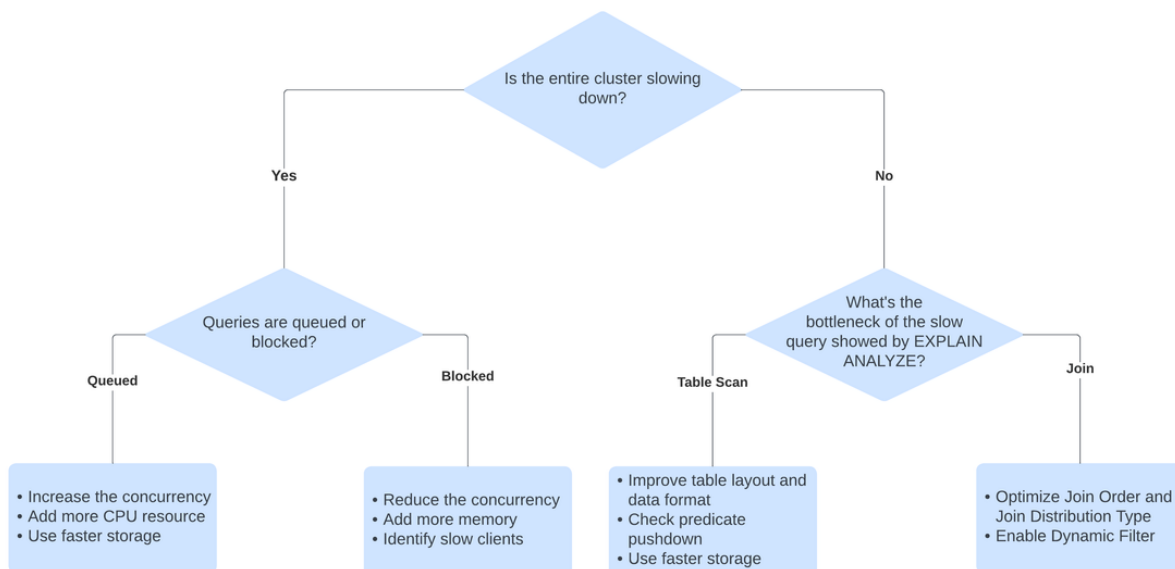
In this chapter, you have learned the fundamentals of Trino's architecture and key concepts for optimization. This foundation is crucial for grasping the best practices and strategies presented in the next chapter.

CHAPTER

02

Best Practices for Trino Optimization

In this chapter, we will discuss various best practices and strategies for Trino performance tuning, ensuring optimal query performance and resource utilization.



The flow chart presented above outlines the recommended optimization process. Make sure that you structure your thoughts and follow these steps before digging into the specific tuning practices.

To begin, you need to determine whether the entire cluster is slowing down or if only a few queries are particularly slow.

If the entire cluster is slowing down, it often indicates that resources are insufficient or that resource allocation and maximum concurrency settings are unreasonable. Like many computing engines, Trino is resource-intensive. When resources are lacking, you will observe a decline in the cluster's overall throughput. In such cases, examine the Trino cluster's Web-UI to determine whether there are more blocked queries or more queries in the queue, so you can take appropriate measures.

On the other hand, if only a few specific queries are slow, you can identify the major bottlenecks of those queries using EXPLAIN ANALYZE results, and then optimize against those bottlenecks.

The following sections of this chapter will explore these optimizations and settings in greater detail.

2.1

Optimize Resource Allocation

Ensuring adequate memory allocation is crucial for query performance. It's important to monitor memory usage and adjust memory settings as necessary. Trino allows you to set the maximum memory per query and the maximum concurrent queries to control resource allocation.

Based on the total resource allocated to the Trino cluster, you will need to figure out a balance between maximum memory per query and the maximum number of concurrent queries. When adjusting the parameters for max concurrency and max memory, you may encounter the following common issues: too many blocked queries or too many queued queries.

If you notice a large number of blocked queries, you could try adding more memory. However, if your total memory resources are limited, consider reducing the maximum number of concurrent queries using Trino's resource group settings. Keep in mind that other factors may contribute to a high number of blocked queries, including, but not limited to:

- Memory allocation deadlock: consider adding `query.low-memory-killer.policy` to Trino's configuration
- Slow clients: some clients may struggle to retrieve results quickly enough

If you see many queued queries, try improving network or storage bandwidth, adding more resources, or increasing maximum concurrency to achieve a balance. Possible solutions include:

- Accelerating I/O or using faster storage if I/O bound
- Adding more CPUs if CPU bound

Some properties mentioned in this section:

Trino Property Name	Suggested Value	Comments
<code>query.max-memory-per-node</code>	JVM heap size / <code>max_concurrent</code>	This value can fluctuate. A higher value can speed up large queries, and a lower value can reduce the risk of memory allocation deadlocks.
<code>query.max-memory</code>	Do not set this limitation if the <code>query.max-memory-per-node</code> has been set properly	
<code>query.low-memory-killer.policy</code>	<code>total-reservation</code> (Only set this value when you're suffering from memory allocation deadlock)	This value will kill the query currently using the most total memory.

After balancing max concurrency and max memory per query, you may need to improve the throughput of the entire Trino cluster by optimizing data storage and query execution. By monitoring and adjusting these settings as needed, you can ensure optimal resource allocation and query performance in your Trino environment.

2.2

Identify Bottlenecks Using EXPLAIN and EXPLAIN ANALYZE

To analyze query plans and performance, use the **EXPLAIN** and **EXPLAIN ANALYZE** commands. **EXPLAIN** shows the query plan, while **EXPLAIN ANALYZE** executes the query and provides additional information about the actual execution, such as the time taken and the number of rows processed.

```
EXPLAIN select * from customers limit 5;
```

OR

```
EXPLAIN ANALYZE select * from customers limit 5;
```

The results of **EXPLAIN** and **EXPLAIN ANALYZE** contain several fragments, as shown below:

```
Fragment 2 [SOURCE]
  CPU: 5.75s, Scheduled: 27.24s, Input: 135937 rows (12.21MB); per task: avg.:
45312.33 std.dev.: 42281.45, Output: 86 rows (8.85kB)
  Output layout: [c_customer_sk, c_customer_id, c_current_cdemo_sk,
c_current_hdemo_sk, c_current_addr_sk, c_first_shipto_date_sk, c_first_sales_date_
  Output partitioning: SINGLE []
  Stage Execution Strategy: UNGROUPED_EXECUTION
  - LimitPartial[5] => [c_customer_sk:integer, c_customer_id:char(16),
c_current_cdemo_sk:integer, c_current_hdemo_sk:integer, c_current_addr_sk:integ
  CPU: 2.00ms (0.03%), Scheduled: 20.00ms (0.07%), Output: 86 rows (8.85kB)
  Input avg.: 3988.68 rows, Input std.dev.: 120.47%
  - TableScan[TableHandle {connectorId='hive',
connectorHandle='HiveTableHandle{schemaName=tpcdssf1alluxio, tableName=customer,
analyzePartitionVa
  CPU: 5.93s (99.93%), Scheduled: 27.75s (99.87%), Output: 123649 rows (10.74MB)
Input avg.: 4385.06 rows, Input std.dev.: 105.48%
LAYOUT: tpcdssf1alluxio.customer{}
c_customer_id := c_customer_id:char(16):1:REGULAR (1:31)
c_email_address := c_email_address:char(50):16:REGULAR (1:31)
c_current_addr_sk := c_current_addr_sk:int:4:REGULAR (1:31)
c_birth_month := c_birth_month:int:12:REGULAR (1:31)
c_current_cdemo_sk := c_current_cdemo_sk:int:2:REGULAR (1:31)
c_customer_sk := c_customer_sk:int:0:REGULAR (1:31)
c_last_name := c_last_name:char(30):9:REGULAR (1:31)
c_salutation := c_salutation:char(10):7:REGULAR (1:31)
c_last_review_date := c_last_review_date:char(10):17:REGULAR (1:31)
c_first_shipto_date_sk := c_first_shipto_date_sk:int:5:REGULAR (1:31)
c_current_hdemo_sk := c_current_hdemo_sk:int:3:REGULAR (1:31)
c_birth_day := c_birth_day:int:11:REGULAR (1:31)
c_preferred_cust_flag := c_preferred_cust_flag:char(1):10:REGULAR (1:31)
c_first_sales_date_sk := c_first_sales_date_sk:int:6:REGULAR (1:31)
c_login := c_login:char(13):15:REGULAR (1:31)
c_first_name := c_first_name:char(20):8:REGULAR (1:31)
c_birth_year := c_birth_year:int:13:REGULAR (1:31)
c_birth_country := c_birth_country:varchar(20):14:REGULAR (1:31)
Input: 135937 rows (12.21MB), Filtered: 9.04%
```

Compared to **EXPLAIN**, **EXPLAIN ANALYZE** offers actual execution statistics (highlighted in the example output above).

By examining the execution statistics of each step, you can identify bottlenecks within the query execution process.

In the example above, the actual CPU cost of Fragment 2 is 5.75s, with the TableScan (a sub-operator of Fragment 4) taking 5.93s, which constitutes the majority of CPU usage in Fragment 2. During performance tuning, you should focus more on the ScanFilterProject part, as it offers greater potential for performance gain.

Different kinds of queries often have different bottlenecks. The following two bottlenecks are most common:

- Table scan: In this case, consider implementing a better table layout, predicate pushdown, or using faster storage.
- Join: In this case, optimize the join distribution type and join order.

We will discuss specific optimization methods in subsequent sections.

2.3

Optimizing File Formats and Table Layouts

Choosing the best file format, compression, partitioning, bucketing, and sorting for your data can significantly impact query performance. Opt for formats like ORC or Parquet, which are columnar and support predicate pushdown. Use partitioning and bucketing to optimize data storage and retrieval, and employ appropriate compression algorithms to balance storage space and query speed.

ORC typically outperforms Parquet in Trino due to a longer history of optimization. However, Parquet has better adoption in the open-source community, and the Trino community is working to improve Parquet file performance.

In some scenarios, the data type of columns might significantly impact optimization. For example, if you use `DOUBLE`, `REAL`, and unorderable data types, the min-max dynamic filter won't work on these columns.

2.3.1

Columnar Data File Formats and Compression

When Trino reads a columnar data file, such as Parquet or ORC, it first reads the file metadata stored at the end of the file in the form of a footer. This metadata determines the structure of the data file and the location of each data section, such as a row group in a Parquet file. The metadata includes information like the schema of the data, the number of rows, the compression codec used, and the offset of each data section within the file.

After reading the metadata, Trino will read the data pages in parallel. It uses multiple threads to read data from different parts of the file simultaneously. The data values for a single column are stored and read together, allowing for more efficient processing and compression. Columnar formats are beneficial for most SQL queries since they don't touch all columns, making it much easier to skip unnecessary columns.

Columnar file formats are also more friendly with predicate pushdown to reduce the rows scanned. The metadata in each data file includes the statistics like the minimum and maximum value for each column. These statistics are collected at two levels: each Parquet file and each sub-section (row-group). When Trino pushes down a predicate to the table scan stage, it utilizes the Parquet file reader to scan the data file. If the query has a filter on a column (e.g., `column1 > 5`), the predicate pushdown helps prune files and row-groups based on the column's statistics.

In earlier versions of Trino, using a flat table column layout in Parquet or ORC reduced the cost of querying nested columns. After Trino version 334, the optimization called dereference pushdown can prune unnecessary data. However, some limitations still exist for dereference pushdown. If you're not seeing the dereference pushdown taking effect by using `EXPLAIN ANALYZE`, you can still flatten your tables to avoid using nested columns.

Columnar data formats also support more efficient compression, which saves storage space and network bandwidth. In columnar formats, data is organized by column, where duplicate values frequently occur, such as date, state, or city. This data organization method enhances compression efficiency. It is highly recommended to convert data to columnar formats like Parquet or ORC.

2.3.2

Partitioning and Bucketing Strategies

Partitioning divides your table into parts based on the value of some columns (partition column or partition key). For Hive tables, each partition's data is stored in a separate folder. If the data scanned for queries with predicates is only on partition columns, Trino will avoid accessing unrelated partitions, thereby saving time on file listing and reading operations.

However, excessive partitions in a single table could lead to performance decline during planning and increased storage pressure due to numerous folders and data files. For instance, using `customer_id` as the partition key may create millions of partitions when you have millions of customers. In such cases, consider employing bucketing, a straightforward form of hash partitioning. Bucket your tables based on one or more columns, utilizing a fixed number of hash buckets.

Here's an example of creating a table with partitioning and bucketing:

```
CREATE TABLE customers (  
  customer_id bigint,  
  created_day date  
)  
WITH (  
  partitioned_by = ARRAY[created_day],  
  bucketed_by = ARRAY['customer_id'],  
  bucket_count = 100  
)
```

In this example, the table is partitioned by the customer's created date and bucketed by the customer's ID, with a fixed total number of 100 buckets.

If you have many buckets, you may see a lot of empty buckets. You can eliminate them by setting the following session key:

```
SET SESSION create_empty_bucket_files = false;
```

2.3.3

Utilizing Materialized Views

A materialized view is a precomputed dataset generated from a query. Unlike regular views, this data is stored for future use. Since the data is already calculated, querying a materialized view becomes faster than executing the original query, especially if the computation is resource-intensive. Implementing materialized views can minimize data scanning, enhance query performance, and optimize resource utilization within your Trino environment. Note that the materialized views only work for the catalogs with the Iceberg Connector.

We recommend using materialized views to speed up expensive aggregation, projection, and selective joins on large tables, especially those that run frequently.

The example below demonstrates how to create a materialized view to store active customers who have logged in within the last 7 days:

```
CREATE MATERIALIZED VIEW active_customers
AS
  SELECT customer_id, created_day
  FROM customers
  WHERE last_login_day >= CURRENT_DATE - INTERVAL '7' DAY;
```

You can manually refresh the materialized view with the following command:

```
REFRESH MATERIALIZED VIEW active_customers
```

The refresh command might be expensive because it re-executes the SQL query and writes the result to storage. We recommend storing materialized views in faster storage, such as Alluxio.

By considering file formats, compression, partitioning, bucketing, and materialized views, you can significantly enhance your Trino query efficiency and overall system performance by reducing the total amount of data scanned.

2.4

Gathering and Using Hive Table Statistics

Collecting and updating statistics for Hive tables is crucial for helping the Trino optimizer make better decisions when planning queries. You can use the following methods to gather and maintain Hive table statistics, ensuring that the optimizer has the most accurate information.

To display the current statistics of a table, use the **SHOW STATS** command:

```
SHOW STATS table_name
```

To gather statistics for a table, use the **ANALYZE** command. Update these statistics periodically to maintain accurate information. It might take a while to recalculate and generate the stats for the entire table, which is why it is advised to generate stats for only one or two columns later on.

```
ANALYZE table_name
```

If you want to refresh the statistics for specific partitions within a Hive table, specify the partition key values (columns) to collect stats for those partitions:

```
ANALYZE table_name WITH (  
  partitions = ARRAY [  
    ARRAY['Dec', '1'], ARRAY['Dec', '2']] )
```

The command above will refresh the stats for the two partitions 'Dec/1' and 'Dec/2' (assuming the table is partitioned by month and day).

For tables with a large number of columns, gathering statistics for all columns can be resource-intensive. In such cases, you can choose to collect statistics only for a subset of columns. We recommend focusing on join keys, grouping keys, and filtering keys:

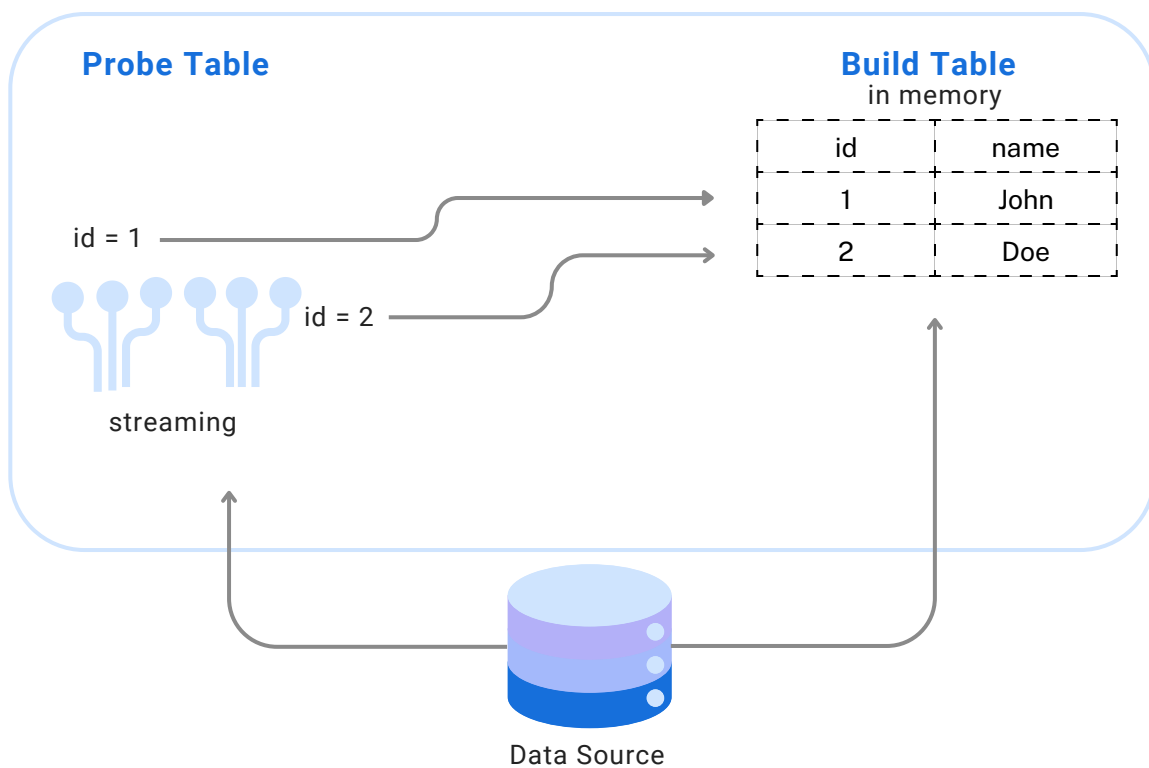
```
ANALYZE table_name WITH (  
  columns = ARRAY['column1', 'column2'])
```

Maintaining accurate statistics can significantly improve query performance. Trino can optimize SQL planning based on these stats, leading to better join orders and join types for queries. This optimization can substantially reduce the amount of data read and exchanged by Trino workers, resulting in more efficient query execution.

2.5 Join Optimization Strategies

Optimizing join operations can significantly improve query performance. By considering join distribution types and join orders, you can use Trino's cost-based optimizer to determine the best join strategy based on table statistics.

Trino always uses a hash join algorithm in its implementation. It is more efficient for Trino to read one table into memory once and reuse its values while scanning the data of the other table. The table read into memory is called the "build table," and the other table is called the "probe table."



The size of each table is important when fitting one of them into memory. To save memory and improve the maximum concurrency of the Trino cluster, it is preferable to choose the smaller table as the build table. This is why table statistics are important for join optimization.

2.5.1

Types of Join Distributions

Trino uses a hash-based join algorithm, which means each Trino worker will build a hash table from the build table. Then, Trino will stream the probe table, and for each row, it will look up the hash table to find matching rows.

There are two types of join distributions:

- **Partitioned:** Each node builds a hash table from only a fraction of the data of the build table.
- **Broadcast:** Each node builds a hash table from all of the data, meaning the build table data is replicated to each node.

Broadcast join can be faster but requires the build table data to fit in the memory of the Trino worker. In this case, join order is crucial, and the smaller table should be selected as the build table.

Partitioned join can be slower as it requires redistributing both tables using a hash of the join key. However, the memory requirement of each node can be lower, as the build table data only needs to fit in the total memory among all Trino workers.

By default, Trino automatically chooses whether to use a partitioned or broadcast join and selects the probe and build sides.

You can change the join distribution strategy by setting the `join_distribution_type` session property:

```
SET SESSION join_distribution_type = AUTOMATIC
```

Possible values are:

AUTOMATIC	Join distribution type is determined automatically for each join
BROADCAST	Broadcast join distribution is used for all joins
PARTITIONED	Partitioned join distribution is used for all joins

You may want to adjust the maximum memory Trino workers can use to build the hash table for broadcast joins:

```
SET SESSION join_max_broadcast_table_size = 100M
```

A higher value might improve performance because a larger build table can also use broadcast join, but a lower value can improve the maximum concurrency of the Trino cluster.

2.5.2

Optimizing Join Orders

A proper join order can significantly reduce the amount of data read from external storage and the amount of data exchanged among workers.

By default, Trino uses cost-based join enumeration, meaning it uses table statistics (collected in the previous section) to estimate the costs for different join orders and automatically picks the join order with the lowest computed costs.

You can also specify the join reordering strategy using the following session key:

```
SET SESSION join_reordering_strategy = AUTOMATIC
```

The default value, "AUTOMATIC," automatically reorders the join based on the computed cost.

Other choices for join reordering strategy are:

ELIMINATE_CROSS_JOINS	Eliminate unnecessary cross joins
NONE	Purely syntactic join order

ELIMINATE_CROSS_JOINS reorders joins to eliminate cross joins, where possible, and otherwise maintains the original query order.

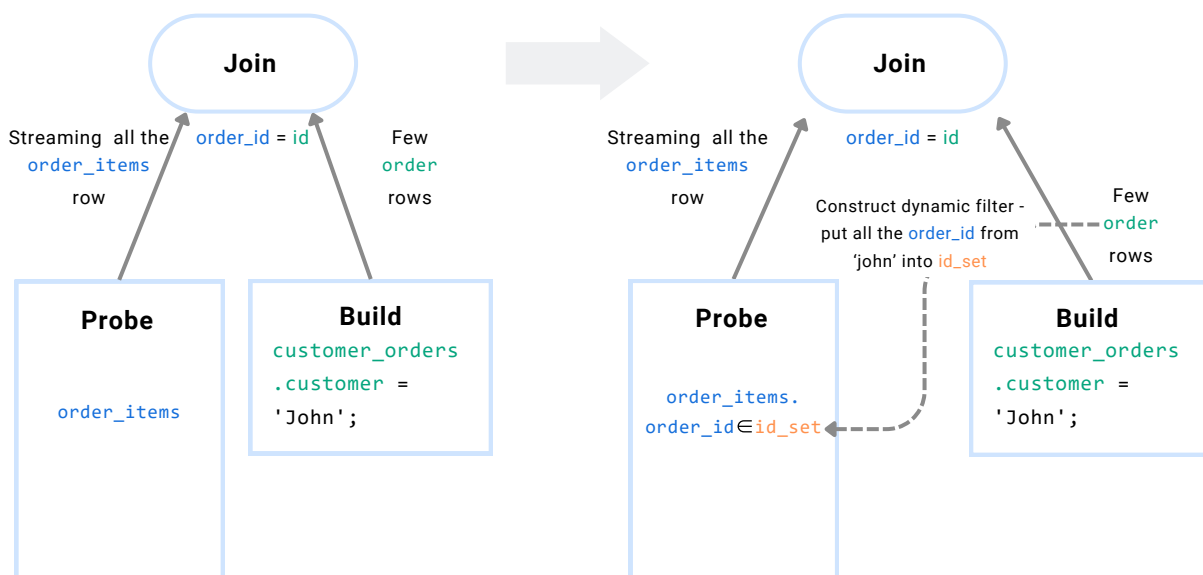
NONE maintains the order the tables are listed in the query.

2.5.3

Implementing Dynamic Filtering

In highly-selective join scenarios (common in ad hoc and dashboard use cases), most rows read from the probe table are not needed and will be ignored during the join, since they don't match the join criteria.

```
SELECT * FROM order_items JOIN customer_orders
ON order_items.order_id = customer_orders.id
WHERE customer_orders.customer = 'John';
```



When Trino uses a broadcast join, each worker collects values from the build table, and after its input is over, applies a dynamic filter generated by the build side to the table scan of the probe side. This dynamic filter can significantly reduce the rows read on the probe side.

By doing so, dynamic filtering can help improve query performance by reducing the amount of data processed during join operations.

Dynamic filtering is enabled by default. If you want to disable it, or if your Trino administrator has not enabled it, you can change it using the session key:

```
SET SESSION enable_dynamic_filtering = false
```

By implementing these join optimization strategies, you can significantly improve the efficiency and performance of your Trino queries. Remember to consider join distribution types and join orders while using Trino's cost-based optimizer, and to leverage dynamic filtering when appropriate.

2.6

Data Caching for Trino

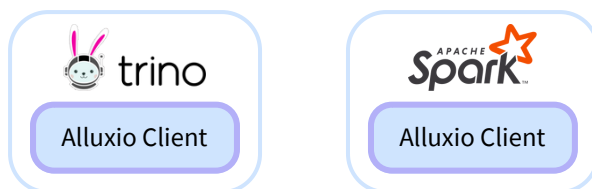
When your data is remote from your Trino clusters or you need to share data between Trino and other compute engines, having a cache for remote data read and data sharing becomes essential. Caching can significantly reduce the latency of data sources when your data is stored in another region.

Utilizing a caching layer can offer several optimizations to Trino queries:

- Enhancing query performance by reducing I/O access latency while co-locating with Trino
- Accelerating queries when reading remote data across datacenters, regions, and clouds
- Serving as a shared cache between Trino workers, across Trino clusters, and other compute engines like Apache Spark
- Providing resilient caching, allowing you to save money using spot instances without losing cached data

Alluxio is a distributed caching layer that can be used between Trino and data sources to improve I/O performance. By caching data closer to the Trino workers, Alluxio reduces the latency of data access and alleviates pressure on the underlying storage system. The architecture diagram below illustrates how Alluxio caching works.

Application



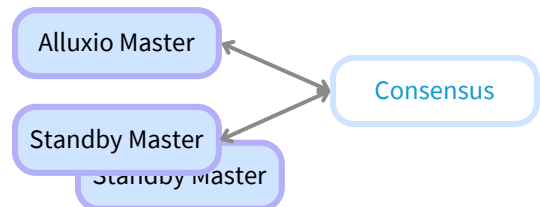
Alluxio Service



Data Service



Metadata Service



Persist Storage



S3 region-us-east 1

Alluxio protects against additional costs and performance degradation, particularly when using S3 and underlying storage. This is because Trino utilizes the HTTPS protocol to fetch data files stored in your S3 bucket. As mentioned previously, when predicate pushdown is effective for columnar data formats like Parquet and ORC, it skips numerous data pieces in the file, potentially causing read fragmentation. This fragmentation increases the number of S3 API calls by 100 to 1000 times. However, by placing Alluxio between Trino and S3, all read requests will be directed to Alluxio. With an HDFS-compatible client and a buffer, Alluxio mitigates the impact of read fragmentation.

To get started with Alluxio caching for Trino, follow this 5-minute tutorial (step-by-step instructions and video): <https://www.alluxio.io/blog/get-started-with-trino-and-alluxio-in-5-minutes/>

2.7

Recommended JVM Configuration Settings

Tuning the JVM configuration settings can help improve Trino's performance and resource utilization. Adjust settings such as heap size, garbage collection options, and JVM flags to optimize Trino for your specific environment and workload.

Applying these best practices and tuning strategies can significantly improve the performance and efficiency of your Trino queries. Experiment with different settings and optimizations to find the best configuration for your use case.

JVM argument name	Suggested Value	Comments
-Xmx	70% ~ 85% of the physical memory of the server.	Highly recommended if Trino is the only major process running on the server
-XX:InitialRAMPercentage=	80.0	The initial heap size of JVM will be bound to 80% of your physical memory
-XX:MaxRAMPercentage=	80.0	The maximum heap size of JVM will be bound to 80% of your physical memory
-XX:-G1UsePreventiveGC	total-reservation (Only set this value when you're suffering from memory allocation deadlock)	Disable the preventive GC to prevent too many objects from being promoted to the old gen for better GC efficiency

You will not need both -Xmx and MaxRAMPercentage in the JVM config, but mostly using MaxRAMPercentage could prevent your Trino process from getting killed by the OOM killer of the operating systems, especially when you're running Trino in a container.

It is not recommended to assign all the physical memory to Trino because some memory should be reserved for the operating system. Additionally, some dependencies, such as the Alluxio client, may use off-heap memory to improve performance. Note that off-heap memory usage won't be counted in the heap usage of JVM.

Chapter 2 Summary

In this chapter, you have learned the tuning steps and seven best practices for Trino optimization. By following these best practices and tuning strategies, you can significantly improve the performance and efficiency of your Trino queries, allowing you to tailor configurations for your specific environment and workload.

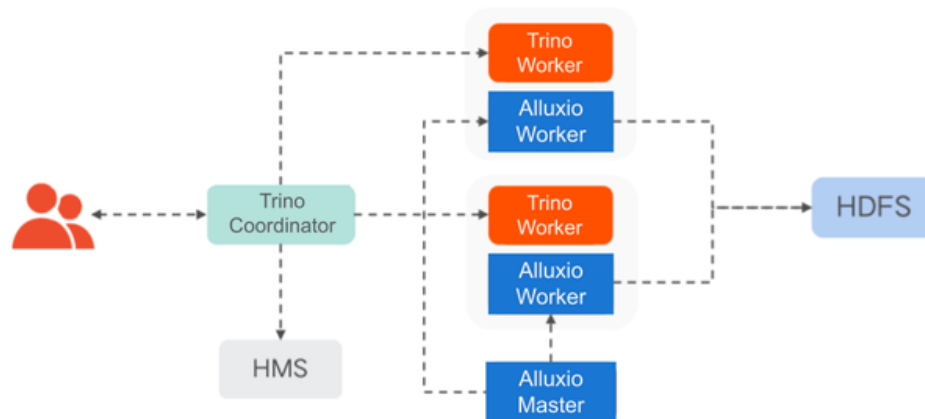
CHAPTER 03 Real-World Case Studies: Optimizing Trino with Caching

In this chapter, you will learn about real-world case studies that showcase the successful implementation of Trino optimization techniques, specifically focusing on caching solutions. By examining the experiences of Shopee and Razorpay, you will gain valuable insights into how these companies tackled challenges in their large-scale deployments and achieved significant improvements in query performance and resource utilization.

3.1

Shopee: Enhancing Query Performance in Large-scale Deployments

Shopee, the leading e-commerce platform in Southeast Asia, faced a significant challenge in running Trino on top of their large-scale HDFS cluster while maintaining consistent performance. Shopee's large-scale Trino deployment includes thousands of Trino nodes, billions of files, and hundreds of thousands of queries per day.



To address these challenges, Shopee adopted the Alluxio as the caching solution and implemented a co-located architecture of Trino and Alluxio, with Trino worker and Alluxio worker on the same machine, and HDFS mounted in Alluxio. Data in HDFS is cached into Alluxio after the first-time access, and Trino accesses the data from Alluxio instead of HDFS.

Other optimizations include adding tags into the partitions property stored in HMS, customizing the cache policy, and allowing Trino queries to access HDFS directly according to tags. They also implemented a CacheManager to manage these changes better.

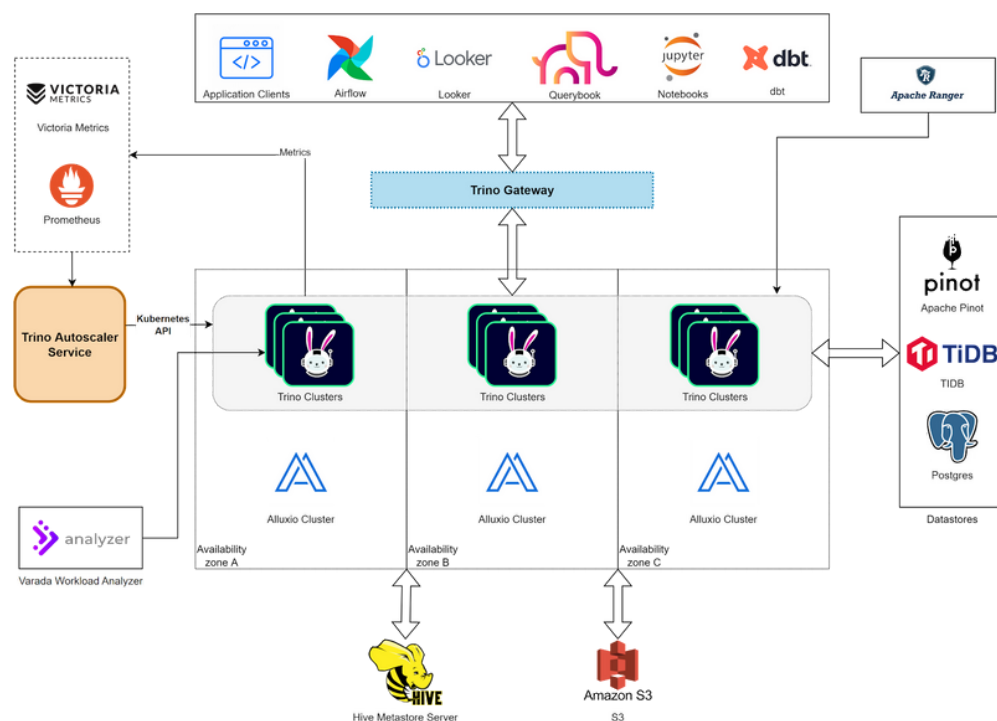
By integrating Alluxio as a caching layer, Shopee significantly reduced query execution time by 55% compared to querying data directly from HDFS. [Read more here >>.](#)

3.2

Razorpay: Adapting Trino Clusters for Elasticity

Razorpay, a large fintech company in India, has built a self-managed Analytics platform based on Trino and Alluxio on Kubernetes. They set up separate clusters for each use case (BI Dashboards and Reports, ELT Schedules, and Ad Hoc Analysis), which improved resource utilization and provided better SLAs.

Razorpay introduced Alluxio as a caching layer between S3 and Trino, which allowed them to share the cache across different Trino clusters, cache both file data and file metadata, support block-level access APIs, and work well with the containerized environment and Kubernetes deployment.



To further improve the scalability of their Trino clusters, Razorpay designed a custom mechanism leveraging Alertmanager for auto-scaling Trino workers, which allowed them to handle varying workload characteristics, iteratively adjust the cluster size while maintaining SLAs, and minimize query failures during downscaling.

With the implementation of these solutions, Razorpay improved query performance, resource efficiency, and user experience while eliminating the need for manual intervention in scaling clusters. The platform serves 650 daily active users and ~100k queries/day at P90 & P95 query latencies of 60 seconds and 130 seconds respectively with a ~97% success rate. [Learn more here >>](#).

Chapter 3 Summary

In this chapter, we explored real-world case studies of companies that have successfully optimized Trino through caching solutions and other techniques. Shopee and Razorpay, both large-scale organizations, tackled unique challenges to improve their Trino deployments' performance, efficiency, and scalability.

CHAPTER

04

Key Takeaways and Resources for Further Learning

4.1

Key Takeaways

Throughout this ebook, you have learned various best practices and tips for optimizing Trino performance, focusing on identifying bottlenecks, memory management, file formats, table layouts, join optimization, dynamic filtering, and JVM configuration. You have also explored how to use caching to improve query performance in real-world case studies from companies like Shopee and Razorpay.

As we conclude, here are the best practices:

- Optimize resource allocation by adjusting memory and CPU settings to maximize resource utilization and minimize contention.
- Identify bottlenecks using EXPLAIN and EXPLAIN ANALYZE to understand query performance and discover areas for improvement.
- Optimize file formats and table layouts by choosing columnar data file formats and compression, implementing partitioning and bucketing strategies, and utilizing materialized views for faster query execution.
- Gather and use Hive table statistics to help Trino make better decisions during query planning and execution.
- Implement join optimization strategies by understanding the types of join distribution, optimizing join orders, and employing dynamic filtering to improve query performance.
- Leverage data caching for Trino, such as using Alluxio, to accelerate queries, reduce latency, and enable efficient data sharing among Trino workers and other compute engines.
- Configure recommended JVM settings to enhance Trino's performance and resource utilization by adjusting heap size, garbage collection options, and JVM flags.

By leveraging the best practices, strategies, and resources outlined in this ebook, you can achieve optimal query performance and resource utilization.

4.2

Additional Resources

To further enhance your understanding and implementation of the techniques provided by this eBook, you can refer to the following resources:

- Dive deeper into Trino's features, configuration options, and performance tuning recommendations by visiting [Trino's documentation](#)
- Unlocking the full potential of Trino and Alluxio in your data analytics infrastructure:
 - Learn more about the synergy between [Alluxio and Trino](#)
 - Learn more about Alluxio's architecture, deployment, and configuration best practices from [Alluxio's documentation](#)
 - Attend online talks to learn from experts and share your experiences with the community. RSVP for Alluxio webinars and open office hours [here](#)
 - Join 10k+ members in the [Alluxio community slack channel](#) to ask any questions and provide your feedback

References

- [1] R. Sethi, M. Traverso, D. Sundstrom, D. Phillips, W. Xie, Y. Sun, N. Yegitbasi, H. Jin, E. Hwang, N. Shingte et al., “Presto: SQL on everything,” in 2019 IEEE 35th International Conference on Data Engineering (ICDE). IEEE, 2019, pp. 1802–1813.
- [2] Trino official repository @ Github: <https://github.com/trinodb/trino>
- [3] Alluxio | Interactive analytics with Trino and Alluxio: <https://www.alluxio.io/trino/>
- [4] Alluxio Blog | Get Started with Trino and Alluxio in 5 Minutes:
<https://www.alluxio.io/blog/get-started-with-trino-and-alluxio-in-5-minutes/>
- [5] Alluxio Blog | “Data Access as a Service” at Shopee: Using Alluxio to Accelerate Interactive Queries and Enhance Developer Experience with Flexible APIs: <https://www.alluxio.io/blog/data-access-as-a-service-at-shopee-using-alluxio/>
- [6] Alluxio Blog | How Trino and Alluxio Power Analytics at Razorpay:
<https://www.alluxio.io/blog/how-trino-and-alluxio-power-analytics-at-razorpay/>

About the Authors

Dr. Beinan Wang has 15 years of experience in performance optimization and large-scale data processing. He contributes to the Trino project and is a PrestoDB committer. Currently a senior staff engineer at Alluxio, he previously led Twitter's Presto team. Dr. Wang earned his Ph.D. in computer engineering from Syracuse University, specializing in distributed systems verification.

“Special thanks to Brian Olsen, the ex-Trino developer advocate. Our collaboration with Brian on the Trino broadcast and the Trino+Alluxio demo has inspired Hope and me to create something truly useful for Trino lovers.”

Hope Wang has a decade of experience in Data, AI, and Cloud. An open-source contributor to Trino, PrestoDB, and Alluxio, she also holds AWS Certified Solutions Architect – Professional status. She leads technical marketing at Alluxio and previously worked in venture capital and as a Data Architect. earned a BS in Computer Science, a BA in Economics, and an MEng in Software Engineering from Peking University, as well as an MBA from USC.

“We love Trino and hope this eBook is helpful to you. Your feedback is welcome, so please reach out to me at hope.wang@alluxio.com. Looking forward to hearing from you and assisting you in your Trino optimization journey!”

About Alluxio

Alluxio, the developer of the open source data platform, makes it easy to manage your data and serve it from any storage to any compute engine in any environment — on premise, in the cloud, or across clouds. By removing complexities and toil from managing and accessing data infrastructure, Alluxio accelerates and future-proofs your data strategy, delivering performant, accessible, cost-effective, resilient, and secure data applications that power improved outcomes, at any scale. To learn more, contact info@alluxio.com or follow us on LinkedIn, or Twitter.